

Week 8: Using existing (already trained) models and deployment

Agenda for today:

Any leftover questions? About LLMs, or anything else?	1
Agenda for the remainder of the term	1
Discussion of using models	1
Using already trained models.....	2
Taking a look at Bea and Fliss's use of machine learning with ml5.js for Lightwaves project.....	2
Speech recognition using Wave2Vec	2
Hand controlled sketches (pose recognition) in p5 using ml5.js.....	2
Deployment (using Flask)	2

Any leftover questions? About LLMs, or anything else?

Agenda for the remainder of the term

- This week is week 8, deployment/using trained models
- Week 9: clarification of things, Project Brief revisit
- 10, 11 – Project support
- *Easter break: 2 weeks—I'll be on leave, but can be reached!*
- Week 12: last week to catch up on things (21 April)
- Week 13: 28 April = deadline (unsure if I will hold that session, given deadline is at 4pm)

Discussion of using models

So far in the module, we have constantly been training models, but the real objective of course is to use the trained models.

So you might be wondering:

1. Why did we spend so much time training the models then?
2. And how does one actually use these models outside of the Jupyter Notebook files in which we trained them?

Using already trained models

One thing you could remember here is that we did take a look at how to save models using `torch.save` as a `state_dict`. (See Word doc for W4 and W5 on saving and loading models.)

```
torch.save(model.state_dict(), PATH)
```

Taking a look at Bea and Fliss's use of machine learning with ml5.js for Lightwaves project

(I hope it's ml5.js anyway!) The thing that I would really want you to understand is that this is a use of a model that's already been trained.

Speech recognition using Wave2Vec

The code for this is this speech recognition tutorial on the PyTorch website here:

https://docs.pytorch.org/audio/stable/tutorials/speech_recognition_pipeline_tutorial.html

-- we might have to skip this, as at the minute there is a version compatibility issue with it, which is annoying—skip for now, maybe I'll sort it by next week

Hand controlled sketches (pose recognition) in p5 using ml5.js

The code for this is here: it's this p5.js tutorial: <https://p5js.org/tutorials/speak-with-your-hands/>

See also the link in there to HandPose, as well as other cool [ml5.js](#) things!

Deployment (using Flask)

What is deployment? It is essentially that you want to be able to use your model without accessing the file that trains it—ideally in a nice GUI, but at least as a first step through Python.

So what we'll do is we'll use Flask to build an app that can run a model for us.

But first, let's walk through the following simple [overview of model deployment](#) (Codegenes):

<https://www.codegenes.net/blog/deploy-pytorch-model/>

Next, we'll take a look at using Flask.

Flask 1: The basics of Flask

The basic way Flask works is the following: Create a file called `app.py`, in which you put the following code:

```
from flask import Flask
app = Flask(__name__)
```

```
@app.route('/')
def hello():
    return 'Hello World!'
```

You can run this file from your Terminal by typing `python app.py`

```
$ FLASK_ENV=development FLASK_APP=app.py flask run
```

Then it will open up on a live server (not accessible to the public) and, if you go to that location, you will see printed 'Hello World!'. This is not so exciting, but with different methods, we can get an inference from the model. The basic structure of that is going to be the same:

- You'll still have an `app.py` (or whatever you want to call it)
- But it'll look a bit different + it'll integrate the model + it'll prepare for you to post correctly formatted inputs to the model
- And the way that you access it will be a bit different: you won't go to the URL, but rather you'll type into the Terminal (for me, outside of VS Code):
`curl -X POST -F "file=@FILENAME.jpg" http://127.0.0.1:5000/predict`

Let's take a look at an example of this in the next section.

Flask 2: Flask with PyTorch

We will draw upon the PyTorch Flask tutorial https://guyuena.github.io/PyTorch-study-Tutorials/intermediate/flask_rest_api_tutorial.html by using the Jupyter Notebook file I shared on Blackboard called `flask-intro-notebook.ipynb`. Download it, and also the image file `kitten.jpg` as well as the `imagenet_class_index.json` file.

1. Start by placing the downloaded 3 files into a new folder on your local harddrive.
2. If you're creating a new venv for this, you'll need a few torch packages and also flask package, but don't worry about that just yet. Most of the packages will show up in the `.ipynb` file, so you'll see if you need to install them.
3. Also, for this one you all don't need to use/run Jupyter Notebook, I'm just sharing this file because it's good for teaching purposes. We will be building an `app.py` file anyway.
4. Follow my walk-through of this file to understand what's going on.

5. Then, we compile our app.py together
6. Once it's done, you can run it using python app.py
When it's running, try posting the image to it by typing, in the Terminal (for me, outside the VS Code terminal):
`curl -X POST -F "file=@kitten.jpg" http://127.0.0.1:5000/predict`
7. Now get some other images to test it out as well!

Note that the PyTorch Flask tutorial has some really nice further reading, e.g. a demo on [how to make a nice UI for inference](#) with ([source code](#)) and also [on deploying in production mode](#), which is a better idea than to stay in developer mode.

Further Deployment References:

General Flask guidance: <https://flask.palletsprojects.com/en/stable/>

A nice and simple, easy to follow, yet comprehensive Medium post on deployment of PyTorch using Flask: <https://medium.com/@ugamakelechi501/deploying-pytorch-models-with-flask-from-training-to-web-api-51b7e0679ebf>

Pair this with the slightly more detailed guide (esp. on Flask) here (CompileNRun): <https://www.compilenrun.com/docs/library/pytorch/pytorch-model-saving/pytorch-model-deployment/>

A different environment from Flask is TorchServe, here is the Geeks description of that: <https://www.geeksforgeeks.org/deep-learning/deploying-pytorch-models-with-torchserve/>